Lab Assignment 9: Data Management Using `pandas`, Part 2

DS 6001

Instructions

Please answer the following questions as completely as possible using text, code, and the results of code as needed. Format your answers in a Jupyter notebook. To receive full credit, make sure you address every part of the problem, and make sure your document is formatted in a clean and professional way.

One of the biggest differences between using SQL and using pandas is the context around the data provenance. If you are using SQL, you are probably working with a relational database with a carefully constructed schema, in third (or a higher level) normal form, curated and organized to prevent time consuming and confusing data quality problems. If you are using pandas to work with raw data files, there are absolutely no guarantees regarding the quality of the data. You might be dealing with unnecessary columns, irrelevant rows, summary statistics included as additional observations, inscrutable column names, columns that should be rows and rows that should be columns, differences in the coverage of cases across data files, or differences in the ways the same observation is identified. You will have no choice but to contend with these issues as the clean, analytic data you generate at the end of all this wrangling may be exactly what you may need.

In this lab, we are going to contend with all of these problems using pandas. We are going to build the Country Analysis Relational DataBase (which we will call the C.A.R.D.B. or the "Cardi B"):



We will be collecting data from two sources. First, we will use open data from the World Bank's Sovereign Environmental, Social, and Governance (ESG) Data project. The ESG data reports data from every country in the world yearly since 1960 on a wide variety of topics including education, health, and economic factors within the countries. Second, we will use data on the quality and democratic character of countries' governments as reported by the Varieties of Democracy (V-Dem) project at the University of Notre Dame. By using both data sources, we can conduct analyses to see whether democratic openness leads to better societal outcomes for countries. We can also write queries to capture a wide range of information on countries' political parties, tax systems, and banking industries, for example. Or as Cardi B would say, "You in the club just to party, I'm there, I get paid a fee. I be in and out them banks so much, I know they're tired of me."

Problem 0

Import the following packages:

In [1]:
```
import numpy as np
import pandas as pd
import requests
import os
import io
import zipfile
```

Both the World Bank and V-Dem store their data in zipped directories containing CSV files. Download the World Bank data into your current working directory by running the following code:

In [2]:
```
url = 'https://databank.worldbank.org/data/download/ESG_CSV.zip'
r = requests.get(url)
```

```
z = zipfile.ZipFile(io.BytesIO(r.content))
z.extractall()
```

And download the V-Dem data by running:

In [3]:
```
url = 'https://v-dem.net/media/datasets/V-Dem-CY-FullOthers-v15_csv.zip'
r = requests.get(url)
z = zipfile.ZipFile(io.BytesIO(r.content))
z.extractall()
```

After you've run this code successfully once, the files you need will be in your working directory and you should save time by switching these cells from "code" to "raw", or commenting out the code, so that they don't run again if you restart the kernel.

You will only need two of the files you've downloaded. Load the 'V-Dem-CY-Full+Others-v15.csv' file as `vdem` and the 'ESGCSV.csv' file as `wb`.

In [4]:
```
vdem = pd.read_csv('data/raw/V-Dem-CY-Full+Others-v15.csv', low_memory=False
wb = pd.read_csv('data/raw/ESGCSV.csv')
```

Problem 1

First, let's focus on the `vdem` data. Use `pandas` methods to perform the following tasks:

Part a

Keep only the 'country_text_id', 'country_name', 'year', 'v2x_polyarchy', and 'v2peedueq' columns. [4 points]

In [5]:
```
columns = [
    "country_text_id",
    "country_name",
    "year",
    "v2x_polyarchy",
    "v2peedueq"
]

vdem = vdem[columns].copy()
vdem.head()
```

Out[5]:

	country_text_id	country_name	year	v2x_polyarchy	v2peedueq
0	MEX	Mexico	1789	0.028	NaN
1	MEX	Mexico	1790	0.028	NaN
2	MEX	Mexico	1791	0.028	NaN
3	MEX	Mexico	1792	0.028	NaN
4	MEX	Mexico	1793	0.028	NaN

Part b

Use the `.query()` method to keep only the rows in which year is greater than or equal to 1960 and less than or equal to 2023 (the World Bank data only covers years between 1960 and 2023, filtering rows here will help us with joining these two dataframes later). [4 points]

In [6]:
```
vdem = vdem.query("1960 <= year <= 2023").copy()
vdem.head()
```

Out[6]:

	country_text_id	country_name	year	v2x_polyarchy	v2peedueq
171	MEX	Mexico	1960	0.231	-1.445
172	MEX	Mexico	1961	0.232	-1.445
173	MEX	Mexico	1962	0.232	-1.445
174	MEX	Mexico	1963	0.232	-1.445
175	MEX	Mexico	1964	0.231	-1.445

Part c

Rename 'country_text_id' to 'country_code', 'country_name' to 'country_name_vdem', 'v2x_polyarchy' to 'democracy', and 'v2peedueq' to 'educational_equality'. [4 points]

In [7]:
```
columns = {
  "country_text_id": "country_code",
    "country_name": "country_name_vdem",
    "v2x_polyarchy": "democracy",
    "v2peedueq": "educational_equality"
}

vdem = vdem.rename(columns=columns)
vdem.head()
```

Out[7]:

	country_code	country_name_vdem	year	democracy	educational_equality
171	MEX	Mexico	1960	0.231	-1.445
172	MEX	Mexico	1961	0.232	-1.445
173	MEX	Mexico	1962	0.232	-1.445
174	MEX	Mexico	1963	0.232	-1.445
175	MEX	Mexico	1964	0.231	-1.445

Part d

Sort the rows by 'country_code' and 'year' in ascending order. [4 points]

In [8]:
```
vdem = vdem.sort_values(by=["country_code", "year"]).reset_index(drop=True)
vdem.head()
```

Out[8]:

	country_code	country_name_vdem	year	democracy	educational_equality
0	AFG	Afghanistan	1960	0.082	-1.203
1	AFG	Afghanistan	1961	0.083	-1.203
2	AFG	Afghanistan	1962	0.083	-1.203
3	AFG	Afghanistan	1963	0.086	-1.203
4	AFG	Afghanistan	1964	0.139	-1.046

Problem 2

Next focus on the World Bank `wb` dataset 'ESGData.csv'. Use `pandas` methods to perform the following tasks:

Part a

Keep only the columns named 'Country Code', 'Country Name', and 'Indicator Code', or begin with '19' or '20'. (Don't type in all the years individually. Instead, use code that finds all columns that begin '19' or '20'.) [4 points]

In [9]:
```
columns = ["Country Code", "Country Name", "Indicator Code"]

year_columns = [
    col for col in wb.columns
    if col.startswith(("19", "20"))
]

wb = wb[columns + year_columns].copy()
wb.head()
```

Out[9]:

	Country Code	Country Name	Indicator Code	1960	1961	1962	1963	1964	1965	196
0	ARB	Arab World	EG.CFT.ACCS.ZS	NaN	NaN	NaN	NaN	NaN	NaN	Nal
1	ARB	Arab World	EG.ELC.ACCS.ZS	NaN	NaN	NaN	NaN	NaN	NaN	Nal
2	ARB	Arab World	EG.ELC.ACCS.RU.ZS	NaN	NaN	NaN	NaN	NaN	NaN	Nal
3	ARB	Arab World	NY.ADJ.DRES.GN.ZS	NaN	NaN	NaN	NaN	NaN	NaN	Nal
4	ARB	Arab World	NY.ADJ.DFOR.GN.ZS	NaN	NaN	NaN	NaN	NaN	NaN	Nal

5 rows × 69 columns

Part b

Rename 'Country Code' to'country_code', 'Country Name' to 'country_name_wb', and 'Indicator Code' to 'feature'. [4 points]

In [10]:

```
columns={
    "Country Code": "country_code",
    "Country Name": "country_name_wb",
    "Indicator Code": "feature"
}

wb = wb.rename(columns=columns)

wb.head()
```

Out[10]:

	country_code	country_name_wb	feature	1960	1961	1962	1963	19
0	ARB	Arab World	EG.CFT.ACCS.ZS	NaN	NaN	NaN	NaN	N
1	ARB	Arab World	EG.ELC.ACCS.ZS	NaN	NaN	NaN	NaN	N
2	ARB	Arab World	EG.ELC.ACCS.RU.ZS	NaN	NaN	NaN	NaN	N
3	ARB	Arab World	NY.ADJ.DRES.GN.ZS	NaN	NaN	NaN	NaN	N
4	ARB	Arab World	NY.ADJ.DFOR.GN.ZS	NaN	NaN	NaN	NaN	N

5 rows × 69 columns

Part c

Use the `.query()` method to remove the rows in which 'country_name_wb' is equal to one of the entries in the folowing `noncountries` list: [4 points]

In [11]:
```
noncountries = ["Arab World", "Central Europe and the Baltics",
                "Caribbean small states",
                "East Asia & Pacific (excluding high income)",
                "Early-demographic dividend","East Asia & Pacific",
                "Europe & Central Asia (excluding high income)",
                "Europe & Central Asia", "Euro area",
                "European Union","Fragile and conflict affected situations",
                "High income",
                "Heavily indebted poor countries (HIPC)","IBRD only",
                "IDA & IBRD total",
                "IDA total","IDA blend","IDA only",
                "Latin America & Caribbean (excluding high income)",
                "Latin America & Caribbean",
                "Least developed countries: UN classification",
                "Low income","Lower middle income","Low & middle income",
                "Late-demographic dividend","Middle East & North Africa",
                "Middle income",
                "Middle East & North Africa (excluding high income)",
                "North America","OECD members",
                "Other small states","Pre-demographic dividend",
                "Pacific island small states",
                "Post-demographic dividend",
                "Sub-Saharan Africa (excluding high income)",
                "Sub-Saharan Africa",
                "Small states","East Asia & Pacific (IDA & IBRD)",
                "Europe & Central Asia (IDA & IBRD)",
                "Latin America & Caribbean (IDA & IBRD)",
                "Middle East & North Africa (IDA & IBRD)","South Asia",
                "South Asia (IDA & IBRD)",
                "Sub-Saharan Africa (IDA & IBRD)",
                "Upper middle income", "World"]
```

In [12]:
```
wb = wb.query("country_name_wb not in @noncountries").copy()
wb.head()
```

Out[12]:

	country_code	country_name_wb	feature	1960	1961	1962	1963
2240	MEA	Middle East, North Africa, Afghanistan & Pakistan	EG.CFT.ACCS.ZS	NaN	NaN	NaN	NaN
2241	MEA	Middle East, North Africa, Afghanistan & Pakistan	EG.ELC.ACCS.ZS	NaN	NaN	NaN	NaN
2242	MEA	Middle East, North Africa, Afghanistan & Pakistan	EG.ELC.ACCS.RU.ZS	NaN	NaN	NaN	NaN
2243	MEA	Middle East, North Africa, Afghanistan & Pakistan	NY.ADJ.DRES.GN.ZS	NaN	NaN	NaN	NaN
2244	MEA	Middle East, North Africa, Afghanistan & Pakistan	NY.ADJ.DFOR.GN.ZS	NaN	NaN	NaN	NaN

5 rows × 69 columns

Part d

The features in this dataset are given strange and incomprehensible codes such as 'EG.CFT.ACCS.ZS'. Use the `replace_map` dictionary, defined below, to recode all of these values with more descriptive names for each feature. [4 points]

In [13]:
```
replace_map = {
  "AG.LND.AGRI.ZS": "agricultural_land",
  "AG.LND.FRST.ZS": "forest_area",
  "AG.PRD.FOOD.XD": "food_production_index",
  "CC.EST": "control_of_corruption",
  "EG.CFT.ACCS.ZS": "access_to_clean_fuels_and_technologies_for_cooking",
  "EG.EGY.PRIM.PP.KD": "energy_intensity_level_of_primary_energy",
  "EG.ELC.ACCS.ZS": "access_to_electricity",
  "EG.ELC.COAL.ZS": "electricity_production_from_coal_sources",
  "EG.ELC.RNEW.ZS": "renewable_electricity_output",
  "EG.FEC.RNEW.ZS": "renewable_energy_consumption",
  "EG.IMP.CONS.ZS": "energy_imports",
  "EG.USE.COMM.FO.ZS": "fossil_fuel_energy_consumption",
  "EG.USE.PCAP.KG.OE": "energy_use",
  "EN.ATM.CO2E.PC": "co2_emissions",
  "EN.ATM.METH.PC": "methane_emissions",
  "EN.ATM.NOXE.PC": "nitrous_oxide_emissions",
  "EN.ATM.PM25.MC.M3": "pm2_5_air_pollution",
  "EN.CLC.CDDY.XD": "cooling_degree_days",
  "EN.CLC.GHGR.MT.CE": "ghg_net_emissions",
  "EN.CLC.HEAT.XD": "heat_index_35",
  "EN.CLC.MDAT.ZS": "droughts",
  "EN.CLC.PRCP.XD": "maximum_5-day_rainfall",
  "EN.CLC.SPEI.XD": "mean_drought_index",
  "EN.MAM.THRD.NO": "mammal_species",
```

```
  "EN.POP.DNST": "population_density",
  "ER.H2O.FWTL.ZS": "annual_freshwater_withdrawals",
  "ER.PTD.TOTL.ZS": "terrestrial_and_marine_protected_areas",
  "GB.XPD.RSDV.GD.ZS": "research_and_development_expenditure",
  "GE.EST": "government_effectiveness",
  "IC.BUS.EASE.XQ": "ease_of_doing_business_rank",
  "IC.LGL.CRED.XQ": "strength_of_legal_rights_index",
  "IP.JRN.ARTC.SC": "scientific_and_technical_journal_articles",
  "IP.PAT.RESD": "patent_applications",
  "IT.NET.USER.ZS": "individuals_using_the_internet",
  "NV.AGR.TOTL.ZS": "agriculture",
  "NY.ADJ.DFOR.GN.ZS": "net_forest_depletion",
  "NY.ADJ.DRES.GN.ZS": "natural_resources_depletion",
  "NY.GDP.MKTP.KD.ZG": "gdp_growth",
  "PV.EST": "political_stability_and_absence_of_violence",
  "RL.EST": "rule_of_law",
  "RQ.EST": "regulatory_quality",
  "SE.ADT.LITR.ZS": "literacy_rate",
  "SE.ENR.PRSC.FM.ZS": "gross_school_enrollment",
  "SE.PRM.ENRR": "primary_school_enrollment",
  "SE.XPD.TOTL.GB.ZS": "government_expenditure_on_education",
  "SG.GEN.PARL.ZS": "proportion_of_seats_held_by_women_in_national_parliamer
  "SH.DTH.COMM.ZS": "cause_of_death",
  "SH.DYN.MORT": "mortality_rate",
  "SH.H2O.SMDW.ZS": "people_using_safely_managed_drinking_water_services",
  "SH.MED.BEDS.ZS": "hospital_beds",
  "SH.STA.OWAD.ZS": "prevalence_of_overweight",
  "SH.STA.SMSS.ZS": "people_using_safely_managed_sanitation_services",
  "SI.DST.FRST.20": "income_share_held_by_lowest_20pct",
  "SI.POV.GINI": "gini_index",
  "SI.POV.NAHC": "poverty_headcount_ratio_at_national_poverty_lines",
  "SI.SPR.PCAP.ZG": "annualized_average_growth_rate_in_per_capita_real_surve
  "SL.TLF.0714.ZS": "children_in_employment",
  "SL.TLF.ACTI.ZS": "labor_force_participation_rate",
  "SL.TLF.CACT.FM.ZS": "ratio_of_female_to_male_labor_force_participation_ra
  "SL.UEM.TOTL.ZS": "unemployment",
  "SM.POP.NETM": "net_migration",
  "SN.ITK.DEFC.ZS": "prevalence_of_undernourishment",
  "SP.DYN.LE00.IN": "life_expectancy_at_birth",
  "SP.DYN.TFRT.IN": "fertility_rate",
  "SP.POP.65UP.TO.ZS": "population_ages_65_and_above",
  "SP.UWT.TFRT": "unmet_need_for_contraception",
  "VA.EST": "voice_and_accountability",
  "EN.CLC.CSTP.ZS": "coastal_protection",
  "SD.ESR.PERF.XQ": "economic_and_social_rights_performance_score",
  "EN.CLC.HDDY.XD": "heating_degree_days",
  "EN.LND.LTMP.DC": "land_surface_temperature",
  "ER.H2O.FWST.ZS": "freshwater_withdrawal",
  "EN.H2O.BDYS.ZS": "water_quality",
  "AG.LND.FRLS.HA": "tree_cover_loss",
}
```

In [14]:
```
wb["feature"] = wb["feature"].replace(replace_map)
wb.head()
```

Out[14]:

	country_code	country_name_wb	featur
2240	MEA	Middle East, North Africa, Afghanistan & Pakistan	access_to_clean_fuels_and_technologies_for_coo.
2241	MEA	Middle East, North Africa, Afghanistan & Pakistan	access_to_electricit
2242	MEA	Middle East, North Africa, Afghanistan & Pakistan	EG.ELC.ACCS.RU.Z
2243	MEA	Middle East, North Africa, Afghanistan & Pakistan	natural_resources_depletio
2244	MEA	Middle East, North Africa, Afghanistan & Pakistan	net_forest_depletio

5 rows × 69 columns

Problem 3

The `wb` dataset is strangely organized. The features are stored in the rows, when typically we would want these features to be columns. Also, years are stored in columns, when typically we would want years to be represented by different rows. We can repair this structure by reshaping the data.

Part a

First, use `pd.melt()` to reshape the data to turn the columns that refer to years into rows. [8 points]

In [15]:
```
wb = pd.melt(
    wb,
    id_vars=["country_code", "country_name_wb", "feature"],
    var_name="year",
    value_name="value"
)

wb.head()
```

Out[15]:

	country_code	country_name_wb	feature	y
0	MEA	Middle East, North Africa, Afghanistan & Pakistan	access_to_clean_fuels_and_technologies_for_coo...	1
1	MEA	Middle East, North Africa, Afghanistan & Pakistan	access_to_electricity	1
2	MEA	Middle East, North Africa, Afghanistan & Pakistan	EG.ELC.ACCS.RU.ZS	1
3	MEA	Middle East, North Africa, Afghanistan & Pakistan	natural_resources_depletion	1
4	MEA	Middle East, North Africa, Afghanistan & Pakistan	net_forest_depletion	1

Part b

Then rename `variable` to `year`, and reshape the data again by using the `.pivot_table()` method to turn the rows that refer to features into columns. Make sure that the country code, country name, and year are included in columns in the dataframe, and not a row index. [8 points]

In [16]:
```
wb = wb.pivot_table(
    index=["country_code", "country_name_wb", "year"],
    columns="feature",
    values="value"
).reset_index()

wb.columns.name = None
wb.head()
```

Out[16]:

	country_code	country_name_wb	year	AG.LND.PFLS.HA	EG.ELC.ACCS.RU.ZS	EN.
0	ABW	Aruba	1960	NaN	NaN	
1	ABW	Aruba	1961	NaN	NaN	
2	ABW	Aruba	1962	NaN	NaN	
3	ABW	Aruba	1963	NaN	NaN	
4	ABW	Aruba	1964	NaN	NaN	

5 rows × 83 columns

Part c

After these reshapes, the year column in the `wb` data frame is stored as a string. Convert this column to an integer data type. [4 points]

In [17]:
```
wb["year"] = wb["year"].astype(int)
wb["year"].dtype
```

Out[17]:
```
dtype('int64')
```

Problem 4

Next we will merge the `wb` data frame with the `vdem` data frame, matching on the 'country_code' and 'year' columns.

Part a

First, write a sentence stating whether you expect this merge to be one-to-one, many-to-one, one-to-many, or many-to-many, and describe your rationale. [4 points]

I expect a one to one merge because we are joining the datasets using `country_code`, and I expect each combination to appear only once in each table.

Part b

Next, merge the two datasets together in a way that checks whether your expectation is met, and also allows you to see the rows that failed to match. [4 points]

In [18]:
```
cardb = pd.merge(
    wb,
    vdem,
    on=["country_code", "year"],
    how="outer",
    validate="one_to_one",
    indicator=True
)

cardb["_merge"].value_counts()
```

Out[18]:
```
_merge
both          10344
left_only      3871
right_only      385
Name: count, dtype: int64
```

Part c

After this merge, use the `.value_counts()` method to see the total number of observations that were found in both datasets, the number found only in the left dataset, and the number found only in the right dataset. (If you entered the `wb` data frame into

the merge function first, then "left_only" refers to the rows found in the World Bank but not V-Dem, and "right_only" refers to the rows found in V-Dem but not the World Bank.) There should be more than 10,000 rows that matched, but more than 2000 that were found in `wb` but not `vdem`, and more than 400 that were found in `vdem` but not `wb`.

Then conduct two data aggregations to help us investigate why these observations did not match:

- First use `.query()` to keep only the observations that were present in `wb` but not `vdem`. (These are the 'left_only' observations if you typed the World Bank data into the merge function first.) Use `.groupby()` to aggregate the data by both 'country_code' and 'country_name_wb'. Include the minimum and maximum values of 'year' for each country in this aggregated dataframe.
- Then use `.query()` to keep only the observations that were present in `vdem` data but not `wb`. Use `.groupby()` to aggregate the data by both 'country_code' and 'country_name_vdem'. Include the minimum and maximum values of 'year' for each country in this aggregated dataframe. [8 points]

In [19]:
```
cardb["_merge"].value_counts()
```

Out[19]:
```
_merge
both          10344
left_only      3871
right_only      385
Name: count, dtype: int64
```

In [20]:
```
wb_only = (
    cardb.query("_merge == 'left_only'")
    .groupby(["country_code", "country_name_wb"])["year"]
    .agg(["min", "max"])
    .reset_index()
)

pd.set_option("display.max_rows", None)
wb_only
```

Out[20]:

	country_code	country_name_wb	min	max
0	ABW	Aruba	1960	2025
1	AFG	Afghanistan	2024	2025
2	AGO	Angola	2024	2025
3	ALB	Albania	2024	2025
4	AND	Andorra	1960	2025
5	ARE	United Arab Emirates	1960	2025
6	ARG	Argentina	2024	2025
7	ARM	Armenia	1960	2025
8	ASM	American Samoa	1960	2025
9	ATG	Antigua and Barbuda	1960	2025
10	AUS	Australia	2024	2025
11	AUT	Austria	2024	2025
12	AZE	Azerbaijan	1960	2025
13	BDI	Burundi	2024	2025
14	BEL	Belgium	2024	2025
15	BEN	Benin	2024	2025
16	BFA	Burkina Faso	2024	2025
17	BGD	Bangladesh	1960	2025
18	BGR	Bulgaria	2024	2025
19	BHR	Bahrain	2024	2025
20	BHS	Bahamas, The	1960	2025
21	BIH	Bosnia and Herzegovina	1960	2025
22	BLR	Belarus	1960	2025
23	BLZ	Belize	1960	2025
24	BMU	Bermuda	1960	2025
25	BOL	Bolivia	2024	2025
26	BRA	Brazil	2024	2025
27	BRB	Barbados	2024	2025
28	BRN	Brunei Darussalam	1960	2025
29	BTN	Bhutan	2024	2025
30	BWA	Botswana	2024	2025
31	CAF	Central African Republic	2024	2025

	country_code	country_name_wb	min	max
32	CAN	Canada	2024	2025
33	CHE	Switzerland	2024	2025
34	CHL	Chile	2024	2025
35	CHN	China	2024	2025
36	CIV	Cote d'Ivoire	2024	2025
37	CMR	Cameroon	1960	2025
38	COD	Congo, Dem. Rep.	2024	2025
39	COG	Congo, Rep.	2024	2025
40	COL	Colombia	2024	2025
41	COM	Comoros	2024	2025
42	CPV	Cabo Verde	2024	2025
43	CRI	Costa Rica	2024	2025
44	CUB	Cuba	2024	2025
45	CUW	Curacao	1960	2025
46	CYM	Cayman Islands	1960	2025
47	CYP	Cyprus	2024	2025
48	CZE	Czechia	2024	2025
49	DEU	Germany	2024	2025
50	DJI	Djibouti	2024	2025
51	DMA	Dominica	1960	2025
52	DNK	Denmark	2024	2025
53	DOM	Dominican Republic	2024	2025
54	DZA	Algeria	2024	2025
55	ECU	Ecuador	2024	2025
56	EGY	Egypt, Arab Rep.	2024	2025
57	ERI	Eritrea	2024	2025
58	ESP	Spain	2024	2025
59	EST	Estonia	1960	2025
60	ETH	Ethiopia	2024	2025
61	FIN	Finland	2024	2025
62	FJI	Fiji	2024	2025
63	FRA	France	2024	2025

	country_code	country_name_wb	min	max
64	FRO	Faroe Islands	1960	2025
65	FSM	Micronesia, Fed. Sts.	1960	2025
66	GAB	Gabon	2024	2025
67	GBR	United Kingdom	2024	2025
68	GEO	Georgia	1960	2025
69	GHA	Ghana	2024	2025
70	GIB	Gibraltar	1960	2025
71	GIN	Guinea	2024	2025
72	GMB	Gambia, The	2024	2025
73	GNB	Guinea-Bissau	2024	2025
74	GNQ	Equatorial Guinea	2024	2025
75	GRC	Greece	2024	2025
76	GRD	Grenada	1960	2025
77	GRL	Greenland	1960	2025
78	GTM	Guatemala	2024	2025
79	GUM	Guam	1960	2024
80	GUY	Guyana	2024	2025
81	HND	Honduras	2024	2025
82	HRV	Croatia	1960	2025
83	HTI	Haiti	2024	2025
84	HUN	Hungary	2024	2025
85	IDN	Indonesia	2024	2025
86	IMN	Isle of Man	1960	2024
87	IND	India	2024	2025
88	IRL	Ireland	2024	2025
89	IRN	Iran, Islamic Rep.	2024	2025
90	IRQ	Iraq	2024	2025
91	ISL	Iceland	2024	2025
92	ISR	Israel	2024	2025
93	ITA	Italy	2024	2025
94	JAM	Jamaica	2024	2025
95	JOR	Jordan	2024	2025

	country_code	country_name_wb	min	max
96	JPN	Japan	2024	2025
97	KAZ	Kazakhstan	1960	2025
98	KEN	Kenya	2024	2025
99	KGZ	Kyrgyz Republic	1960	2025
100	KHM	Cambodia	2024	2025
101	KIR	Kiribati	1960	2025
102	KNA	St. Kitts and Nevis	1960	2025
103	KOR	Korea, Rep.	2024	2025
104	KWT	Kuwait	2024	2025
105	LAO	Lao PDR	2024	2025
106	LBN	Lebanon	2024	2025
107	LBR	Liberia	2024	2025
108	LBY	Libya	2024	2025
109	LCA	St. Lucia	1960	2025
110	LIE	Liechtenstein	1960	2025
111	LKA	Sri Lanka	2024	2025
112	LSO	Lesotho	2024	2025
113	LTU	Lithuania	1960	2025
114	LUX	Luxembourg	2024	2025
115	LVA	Latvia	1960	2025
116	MAF	St. Martin (French part)	1960	2025
117	MAR	Morocco	2024	2025
118	MCO	Monaco	1960	2025
119	MDA	Moldova	1960	2025
120	MDG	Madagascar	2024	2025
121	MDV	Maldives	2024	2025
122	MEA	Middle East, North Africa, Afghanistan & Pakistan	1960	2025
123	MEX	Mexico	2024	2025
124	MHL	Marshall Islands	1960	2025
125	MKD	North Macedonia	1960	2025
126	MLI	Mali	2024	2025
127	MLT	Malta	2024	2025

	country_code	country_name_wb	min	max
128	MMR	Myanmar	2024	2025
129	MNA	Middle East, North Africa, Afghanistan & Pakis...	1960	2025
130	MNE	Montenegro	1960	2025
131	MNG	Mongolia	2024	2025
132	MNP	Northern Mariana Islands	1960	2024
133	MOZ	Mozambique	2024	2025
134	MRT	Mauritania	2024	2025
135	MUS	Mauritius	2024	2025
136	MWI	Malawi	2024	2025
137	MYS	Malaysia	2024	2025
138	NAM	Namibia	2024	2025
139	NCL	New Caledonia	1960	2025
140	NER	Niger	2024	2025
141	NGA	Nigeria	2024	2025
142	NIC	Nicaragua	2024	2025
143	NLD	Netherlands	2024	2025
144	NOR	Norway	2024	2025
145	NPL	Nepal	2024	2025
146	NRU	Nauru	1960	2025
147	NZL	New Zealand	2024	2025
148	OMN	Oman	2024	2025
149	PAK	Pakistan	2024	2025
150	PAN	Panama	2024	2025
151	PER	Peru	2024	2025
152	PHL	Philippines	2024	2025
153	PLW	Palau	1960	2025
154	PNG	Papua New Guinea	2024	2025
155	POL	Poland	2024	2025
156	PRI	Puerto Rico	1960	2025
157	PRK	Korea, Dem. People's Rep.	2024	2025
158	PRT	Portugal	2024	2025
159	PRY	Paraguay	2024	2025

	country_code	country_name_wb	min	max
160	PYF	French Polynesia	1960	2025
161	QAT	Qatar	2024	2025
162	ROU	Romania	2024	2025
163	RUS	Russian Federation	2024	2025
164	RWA	Rwanda	2024	2025
165	SAU	Saudi Arabia	2024	2025
166	SDN	Sudan	2024	2025
167	SEN	Senegal	2024	2025
168	SGP	Singapore	2024	2025
169	SLB	Solomon Islands	2024	2025
170	SLE	Sierra Leone	2024	2025
171	SLV	El Salvador	2024	2025
172	SMR	San Marino	1960	2025
173	SOM	Somalia	2024	2025
174	SRB	Serbia	2024	2025
175	SSD	South Sudan	1960	2025
176	STP	Sao Tome and Principe	2024	2025
177	SUR	Suriname	2024	2025
178	SVK	Slovak Republic	1960	2025
179	SVN	Slovenia	1960	2025
180	SWE	Sweden	2024	2025
181	SWZ	Eswatini	2024	2025
182	SXM	Sint Maarten (Dutch part)	1960	2025
183	SYC	Seychelles	2024	2025
184	SYR	Syrian Arab Republic	2024	2025
185	TCA	Turks and Caicos Islands	1960	2025
186	TCD	Chad	2024	2025
187	TGO	Togo	2024	2025
188	THA	Thailand	2024	2025
189	TJK	Tajikistan	1960	2025
190	TKM	Turkmenistan	1960	2025
191	TLS	Timor-Leste	2024	2025

	country_code	country_name_wb	min	max
192	TMN	Middle East, North Africa, Afghanistan & Pakis...	1960	2025
193	TON	Tonga	1960	2025
194	TTO	Trinidad and Tobago	2024	2025
195	TUN	Tunisia	2024	2025
196	TUR	Turkiye	2024	2025
197	TUV	Tuvalu	1960	2025
198	TZA	Tanzania	2024	2025
199	UGA	Uganda	2024	2025
200	UKR	Ukraine	1960	2025
201	URY	Uruguay	2024	2025
202	USA	United States	2024	2025
203	UZB	Uzbekistan	1960	2025
204	VCT	St. Vincent and the Grenadines	1960	2025
205	VEN	Venezuela, RB	2024	2025
206	VGB	British Virgin Islands	1960	2025
207	VIR	Virgin Islands (U.S.)	1960	2025
208	VNM	Viet Nam	2024	2025
209	VUT	Vanuatu	2024	2025
210	WSM	Samoa	1960	2025
211	XKX	Kosovo	1996	2025
212	YEM	Yemen, Rep.	2024	2025
213	ZAF	South Africa	2024	2025
214	ZMB	Zambia	2024	2025
215	ZWE	Zimbabwe	2024	2025

In [21]:
```
vdem_only = (
    cardb.query("_merge == 'right_only'")
    .groupby(["country_code", "country_name_vdem"])["year"]
    .agg(["min", "max"])
    .reset_index()
)

vdem_only
```

Out[21]:

	country_code	country_name_vdem	min	max
0	DDR	German Democratic Republic	1960	1990
1	HKG	Hong Kong	1960	2023
2	PSE	Palestine/West Bank	1967	2023
3	PSG	Palestine/Gaza	1960	2023
4	SML	Somaliland	1991	2023
5	TWN	Taiwan	1960	2023
6	VDR	Republic of Vietnam	1960	1975
7	XKX	Kosovo	1999	1999
8	YMD	South Yemen	1960	1990
9	ZZB	Zanzibar	1960	2023

Part d

Here's where a deep understanding of the data becomes very important. There are two reasons why an observation may fail to match in a merge. One reason is a difference in spelling. Suppose that South Korea (which is also known as the Republic of Korea) is coded as SKO in the World Bank data and ROK in V-Dem. In this case, we should recode one or the other of SKO and ROK so that they match, otherwise we will lose the data on South Korea. But the second reason why observations might fail to match is due to differences in coverage in the data collection strategy: it is possible that a country wasn't included in one data's coverage, or that certain years for that country were not included. For differences in coverage, there's no way to manipulate the data to match, so we are out of luck and we have to either delete these observations or proceed with missing data from one of the data sources.

Ukraine, and for some reason Seychelles, have no data from the World Bank data for 2023. Please ignore those issues for now. But for the other unmatched rows, take a close look at the two data aggregation tables you generated in part (c), and answer the following questions:

- Do you see any countries that are present in both the unmatched World Bank rows and the unmatched V-Dem rows, but with different spellings?

I didn't notice any countries with different spellings between the two tables. Kosovo appears in both, but not spelling error.

- Do some digging on Wikipedia and other sources on the Internet. What do you think is the primary reason why some countries are present in the V-Dem data but not the

World Bank? (You don't need to describe the reasoning for every country. Just dig until you see a general pattern and describe it here.)

After investigating, I think the main reason is that the V-Dem dataset contains records of countries that no longer exist, such as East Germany, South Yemen and others.

- Do some more digging on Wikipedia and other sources on the Internet. What do you think is the primary reason why some countries are present in the World Bank data but not V-Dem? (You don't need to describe the reasoning for every country. Just dig until you see a general pattern and describe it here.)

Another reason could be that the World Bank covers more countries and territories than V-Dem, especially smaller islands and territories.

[8 points]

Part e

Once you are convinced that all of the unmatched observations are due to differences in the coverage of the data collection strategies of the World Bank and V-Dem, repeat the merge, dropping all unmatched observations. This time there is no need to validate the type of merge, and no need to define a variable to indicate matching. [4 points]

In [22]:
```
cardb = pd.merge(
    wb,
    vdem,
    on=["country_code", "year"],
    how="inner"
)

cardb.head()
```

Out[22]:

	country_code	country_name_wb	year	AG.LND.PFLS.HA	EG.ELC.ACCS.RU.ZS	EN.
0	AFG	Afghanistan	1960	NaN	NaN	
1	AFG	Afghanistan	1961	NaN	NaN	
2	AFG	Afghanistan	1962	NaN	NaN	
3	AFG	Afghanistan	1963	NaN	NaN	
4	AFG	Afghanistan	1964	NaN	NaN	

5 rows × 86 columns

In [23]:
```
cardb.shape
```

Out[23]:
```
(10344, 86)
```

Problem 5

Write code using `pandas` that answers the next two questions:

Part a

Of all countries in the data, since 1960, which countries have the highest and lowest average levels of democratic quality? [10 points]

In [24]:
```
df_1960 = cardb[cardb['year'] >= 1960]

country_avg = (
    df_1960.groupby('country_name_vdem')['democracy']
    .mean()
    .sort_values()
)

print("Lowest average:")
print(country_avg.head(1))

print("\nHighest average:")
print(country_avg.tail(1))
```

```
Lowest average:
country_name_vdem
Saudi Arabia    0.015422
Name: democracy, dtype: float64

Highest average:
country_name_vdem
Denmark    0.911813
Name: democracy, dtype: float64
```

Denmark has had the highest average, while Saudi Arabia has had the lowest.

Part b

The 'educational_equality' index compiled by V-Dem measures the extent to which "high quality basic education guaranteed to all, sufficient to enable them to exercise their basic rights as adult citizens." They use a Bayesian scaling method to create a score for each country in each year that ranges roughly from -4 to 4, where low values of the scale mean that

> Provision of high quality basic education is extremely unequal and at least 75 percent (%) of children receive such low-quality education that undermines their ability to exercise their basic rights as adult citizens.

And high values mean that

> Basic education is equal in quality and less than five percent (%) of children receive such low-quality education that probably undermines their ability to exercise their basic rights as adult citizens.

Use the `pd.cut()` method to create a categorical version of 'educational_equality' with five categories, one from -4 to -2 called "extremely unequal", one from -2 to -.5 called "very unequal", one from -.5 to .5 called "somewhat unequal", one from .5 to 1.5 called "relatively equal", and one for values from 1.5 to 4 called "equal". (By default, the `pd.cut()` method sets `right=True`, which means the bins include their rightmost edges, so a value of exactly -2 will fall within the "extremely unequal" bin. Leave this default in place.)

Then aggregate the data to have one row per category of the new categorical version of "educational_equality". Collapse the following features to the mean with each category of "educational_equality":

- 'gini_index': The GINI index measures the amount of economic inequality in a country. The higher the index, the greater the economic disparity between rich and poor.
- 'poverty_headcount_ratio_at_national_poverty_lines': a measure of the proportion of the population living in poverty

Based on what you see, how would you describe the relationship between educational inequality, economic inequality, and poverty? (Don't worry about complete rigor with respect to causal inference, just describe what you see in this table.)

[10 points]

In [25]:
```
bins = [-4, -2, -0.5, 0.5, 1.5, 4]

labels = [
    'extremely unequal',
    'very unequal',
    'somewhat unequal',
    'relatively equal',
    'equal'
]

cardb['education_category'] = pd.cut(
    cardb['educational_equality'],
    bins=bins,
    labels=labels
)

education_summary = (
    cardb.groupby('education_category', observed=True)[
        ['gini_index', 'poverty_headcount_ratio_at_national_poverty_lines']
    ]
    .mean()
)
```

```
education_summary
```

Out[25]:

education_category	gini_index	poverty_headcount_ratio_at_national_poverty_lines
extremely unequal	41.930435	58.462500
very unequal	45.540125	38.002885
somewhat unequal	43.130165	26.472327
relatively equal	36.469414	21.039370
equal	32.491833	18.180612

Countries with more equal education tend to have lower poverty rates. Poverty decreases as educational equality increases. For economic inequality, the pattern is less clear. The GINI index generally decreases as education becomes more equal, but there are some fluctuations along the way.